National Chung Hsing University

System-on-Chip Design Course

Final Project Proposal

期末專題構想說明書

Porting an Open-Source RV32IM Soft Core (tinyriscv)

onto ZedBoard with Custom Peripheral Extension

將開源 RV32IM 軟核 (tinyriscv) 移植至 ZedBoard

並擴充自訂 Peripheral

SoC Design（系統晶片設計）
吳崇賓副教授
國立中興大學資訊工程研究所

April 22, 2026

Abstract

RISC-V 作為新興開源指令集架構（ISA），近年於嵌入式 SoC 領域快速發展。然而對學生而言，於一學期專題時程內完成一顆可下板運作、可執行 C 程式之 RISC-V SoC 系統，並同時掌握 RTL 設計、bus interconnect、peripheral 整合、C firmware 等完整流程，仍面臨相當之實作門檻。

本專題於 **Avnet ZedBoard（Xilinx Zynq-7020）**平台上建立一顆完整之 RV32IM 32-bit RISC-V SoC 系統，包含 3-stage pipeline CPU 核心、bus interconnect、UART / GPIO / Timer / SPI peripherals、以及**本專題設計之自訂 peripheral IP**（候選：CRC32、4×4 MAC accelerator、Sobel filter）。設計過程將處理跨 FPGA 平台之 clock domain、IO bank voltage、reset polarity 等差異，並於下板環境重新驗證 RV32IM ISA compliance。

預期完成後，ZedBoard 於上電後由 UART 輸出 `"Hello ZedBoard"` 字串、執行 ISA compliance test 並由 LED 指示測試結果、以及示範自訂 peripheral 之 C driver 呼叫流程；同時產出完整之 bring-up 流程紀錄與 bus 擴充驗證案例，作為 Xilinx FPGA 平台 RISC-V SoC 整合之實作參考。

關鍵字：RISC-V、SoC 整合、ZedBoard、FPGA 平台移植、RV32IM

1 Introduction

1.1 Course Context and Platform Selection

本專題為「SoC Design（系統晶片設計）」課程之期末研究專題。該課程於 Lab 1–8 中以 **Avnet ZedBoard（Xilinx Zynq-7020）**為實作平台，累積 Vivado 2019.1 工具流程、Block Design、AXI-Lite 自訂 IP、Xilinx SDK C firmware、interrupt handling、以及 ILA 硬體除錯等實務經驗。延續此平台經驗，本期末專題同樣以 ZedBoard 為實作目標有以下技術優勢：

- **工具鏈延續性**：Vivado 2019.1 WebPACK、Xilinx SDK、USB-JTAG programmer 已於 Lab 階段設定完成，可立即進入 RTL 與系統整合階段，節省環境配置時間。
- **雙架構擴充空間**：Zynq-7020 同時具備 ARM Cortex-A9 PS（Processing System）與 Artix-7 equivalent PL（Programmable Logic），除可於 PL 側部署自行設計之 RISC-V 核心外，亦為 Phase C 之 PS/PL 協同驗證（以 PS 作為 debug host 透過 AXI 控制 PL 上之 RISC-V）保留擴充路徑。
- **豐富板上資源**：ZedBoard 具 8 個 LED、8 個 switches、5 個 push buttons、DDR3 memory、3 組 PMOD、以及 USB-UART，足以對應 RV32IM 軟核之所有 I/O 與 status 指示需求。

1.2 Background and Motivation

處理器架構是數位 IC 設計中整合度最高的應用，牽涉 control path、datapath、hazard detection、interrupt handling、bus interconnect 等核心議題，是檢驗 SoC 設計能力的指標性題目。近年 **RISC-V** 作為新興開源 ISA 快速發展，提供開放、可讀、可擴充之處理器設計範本；相較於單一功能加速器（AI accelerator、video codec），RISC-V SoC 題目能同時訓練**架構思維**與**系統整合**能力，因此選定為本專題之核心題材。

1.3 Rationale for Choosing tinyriscv as the Base Design

在開源 RISC-V 專案中，工業級實作（Rocket Chip、BOOM、CV32E40P）架構複雜度高，不適合在一學期時程內完整掌握。本專題選定 tinyriscv 作為基礎設計，因其明確定位為**教學用**之精簡實作：RV32IM ISA，已通過 riscv-compliance 測試；3-stage pipeline，約 6,000 行可讀性高的 Verilog 2001；內建 UART / GPIO / Timer / SPI / CLINT / JTAG debugger；提供 toolchain 下載連結、Python-based firmware downloader、範例程式與 compliance 測試框架。此特性使其成為「閱讀 $\rightarrow$ 理解 $\rightarrow$ 移植 $\rightarrow$ 擴充」之合適題材，能在合理時程下達成**下板運作**與**深度系統整合**之工程目標。

1.4 Career and Educational Alignment

本專題之實作經驗對團隊成員於數位 IC 產業之多方向發展皆有助益：無論未來朝 **Design Verification（DV）**、**Design Engineer（DE）**、或 **AI-assisted EDA** 發展，皆需對處理器微架構、

RTL 驗證流程、以及 SoC 整合有紮實掌握。透過下板實機驗證與深度筆記之撰寫，能同時建立對 pipeline、hazard、bus interconnect 等議題之實務理解。

2 Related Work and References

2.1 Primary References

本專題之核心參考資源整理如 Table 1：

Table 1: Primary References for the Project.

Resource	URL / Citation	Purpose
tinyriscv GitHub	liangkangnan/tinyriscv	原始碼、toolchain、test suite
Author's Design Blog	CSDN blog post	各模組之設計動機解說
RISC-V ISA Spec.	riscv.org	RV32IM 指令定義
ZedBoard HW User Guide v2.2	Avnet 官方文件	接腳、Bank 電壓、板上資源
Patterson & Hennessy	*Computer Org. & Design* (RISC-V ed.)	Pipeline / hazard / forwarding 理論

2.2 Comparison of Open-Source RISC-V Cores

Table 2 比較幾個主要開源 RISC-V 處理器：

Table 2: Comparison of Open-Source RISC-V Cores.

Core	ISA	Pipeline	Positioning	Adoption
tinyriscv	RV32IM	3-stage	Educational	✓Selected
PicoRV32	RV32IMC	Non-fixed	Minimum area	可讀性略差
CV32E40P	RV32IMFC	4-stage	Industrial	複雜度過高
Rocket Chip	RV64GC	5-stage OoO	Research	Chisel 語言未修

3 System Architecture

3.1 Original tinyriscv Architecture

tinyriscv 內部採用作者自訂之 **RIB bus**，連接 4 個 master 與 6 個 slave，如 Figure 1 所示。CPU 核心採 IF → ID → EX 三級管線，實作 RV32IM 指令集（含 M extension 的乘除法），並透過 CLINT 處理 timer interrupt。

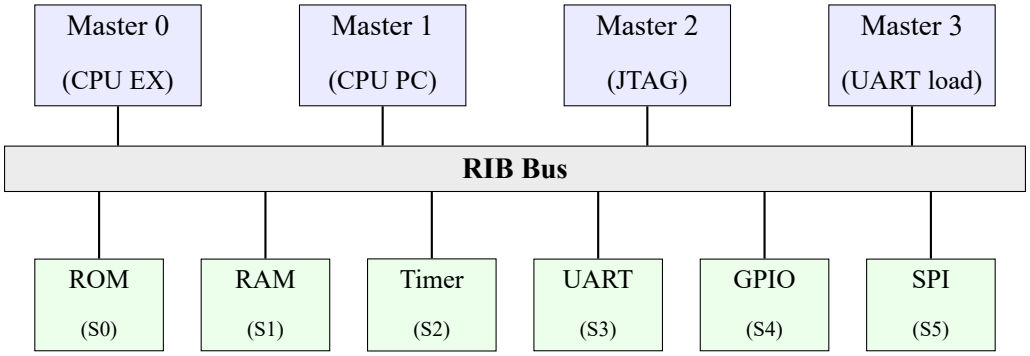


Figure 1: tinyriscv 的 RIB bus interconnect：4 個 master（CPU 執行 / PC、JTAG、UART loader）與 6 個 slave（ROM / RAM / Timer / UART / GPIO / SPI）。

3.2 Board-Level Configuration on ZedBoard

本專題保留 tinyriscv 原樣 RTL，另行撰寫 board-level wrapper（`zedboard_top.v`）將 SoC 頂層訊號對應至 ZedBoard 之硬體資源（Table 3）。

Table 3: ZedBoard Pin Assignment for tinyriscv SoC Top-Level Ports.

tinyriscv Top-Level Port	**ZedBoard Resource**	**Bank / IOSTANDARD**
`clk`	GCLK Y9（100 MHz，內部分頻為 50 MHz）	Bank 13 / LVCMOS33
`rst`	BTNC P16（反相為 active-low）	Bank 34 / LVCMOS18
`uart_tx_pin / rx_pin`	PMOD JA1 / JA2 + USB-UART 轉接	Bank 13 / LVCMOS33
`uart_debug_pin`	SW0 F22	Bank 34 / LVCMOS18
`gpio[1:0]`	LD0 / LD1	Bank 33 / LVCMOS33
`over / succ / halted_ind`	LD2 / LD3 / LD4	Bank 33 / LVCMOS33
JTAG / SPI 各 4 pin	Phase A 不接（tie-off）	—

3.3 Clock Strategy

ZedBoard GCLK 為 100 MHz，但 tinyriscv 原生 UART 之 baud rate divisor（`uart.v`: `BAUD_115200 = 32'h1B8 = 440`）是為 **50 MHz 時脈**計算：

$$\text{divisor} = \frac{50{,}000{,}000}{115{,}200} \approx 434 \approx 440 \tag{1}$$

若直接將 100 MHz 餵給 tinyriscv，實際 baud rate 會變為 57,600，serial terminal 將印出亂碼。本專題於 wrapper 內做 **divide-by-2 分頻**，使內部時脈維持 50 MHz，與 upstream 所有 timing 參數一致；

效能代價僅為 CoreMark 從 2.4 降為 1.2（屬可接受範圍）。此策略之優點為可保持「原樣移植」原則，便於後續升版。

4 Implementation Plan

4.1 Phase A: Direct Porting and Board Bring-up (Baseline Goal)

目標：建立可運作之最小系統，讓 tinyriscv 在 ZedBoard 上成功啟動並執行 hello world 程式。時程如 Table 4。

Table 4: Phase A Implementation Timeline.

Step	Task	Timeline
1	Clone tinyriscv、建立 repo 結構	Completed
2	閱讀 tinyriscv 架構、撰寫 `ARCH_OVERVIEW.md`	Completed
3	Vivado project 建立（target ZedBoard）、撰寫 `zedboard_top.v`	Week 1
4	撰寫 `tinyriscv_zed.xdc`、pin assignment 驗證	Week 1
5	Synthesis / Implementation / Bitstream 生成、timing closure	Week 2
6	下板 + UART firmware 載入 + hello world 驗證	Week 2

4.2 Phase B: Custom Peripheral on RIB Bus (Intermediate Goal)

目標：於 tinyriscv 原生 RIB bus 上新增自行設計之 peripheral IP，展現 SoC 整合能力。候選題目以與 AI 運算相關之 datapath 為優先，時程如 Table 5。

Table 5: Phase B Implementation Timeline.

Step	Task	Timeline
7	Peripheral 選題（候選：CRC32 / 4×4 MAC accelerator / Sobel filter）	Week 3
8	設計 RIB slave wrapper（對齊 upstream uart / gpio / timer 介面規格）	Week 3–4
9	修改 `tinyriscv_soc_top.v` address decode、接入新 slave	Week 4
10	撰寫 C driver，CPU 呼叫 peripheral 並由 UART 印出結果	Week 5
11	對 6 個核心模組（pc_reg、id、ex、regs、clint、rib）撰寫深度筆記	Week 5–6

4.3 Phase C: Stretch Goals (Time-permitting)

- **Zynq PS Integration**：將 tinyriscv 包成 AXI-Lite slave IP，由 Zynq PS 擔任 debug host

- **ASIC-Flow Synthesis**：於系所伺服器（140.120.14.11）以 Synopsys Design Compiler + CBDK v2.1 合成 tinyriscv core，產出 area / timing report

4.4 Research Tasks

本專題執行期間將進行之具體研究與工程工作項目如下：

1. **tinyriscv 原始碼研讀**：完整解析 `rtl/core/`（pc_reg、if_id、id、id_ex、ex、regs、ctrl、clint、div）、`rtl/perips/`（uart、gpio、timer、spi、rom、ram）、`rtl/debug/`（jtag_top、uart_debug）、`rtl/soc/tinyriscv_soc_top.v` 之模組架構、訊號流、以及 RIB bus 之 master/slave 介面協定。
2. **Board-level Wrapper 設計**：撰寫 `zedboard_top.v`，處理 (a) 100 MHz → 50 MHz clock divider 與 `create_generated_clock` 約束；(b) BTNC active-high 到 tinyriscv active-low reset 之極性反相；(c) JTAG / SPI 於 Phase A 之介面 tie-off。
3. **XDC 約束撰寫**：針對 ZedBoard 各 Bank（Bank 13 / 33 = LVCMOS33，Bank 34 = LVCMOS18）正確標示 IOSTANDARD、pin package location、時脈週期，並對照 ZedBoard Hardware User Guide v2.2 進行 cross-check。
4. **Synthesis 與 Implementation 收斂**：Vivado 2019.1 synthesis / implementation / bitstream 流程；確認 timing close（WNS ≥ 0）、檢視 utilization report（LUT / FF / BRAM）、處理 critical warning。
5. **下板 Bring-up 與 Debug**：UART firmware loader 驗證、Serial terminal 接收 `"Hello ZedBoard"` 輸出、LED 狀態確認；所有異常現象與解法紀錄於 `BRINGUP_LOG.md`。
6. **ISA Compliance 驗證**：執行 tinyriscv 自帶之 `tests/riscv-compliance/` 於下板環境，確認 RV32IM 指令集行為正確。
7. **自訂 Peripheral 設計（Phase B）**：選定 CRC32 / 4×4 MAC / Sobel filter 三者之一，設計 RIB slave 介面（對齊 `uart.v` 之 `we_i / addr_i / data_i / data_o` 規格），擴充 `rib.v` 之 address decode，撰寫對應 C driver。
8. **模擬驗證與波形分析**：以 Icarus Verilog 建立 testbench、用 GTKWave 分析 pipeline 關鍵 instruction 之 critical path 波形。
9. **教學文件撰寫**：產出 `ARCH_OVERVIEW.md`、`PORTING_GUIDE.md`、`BRINGUP_LOG.md`、`RTL_WALKTHROUGH.md`（核心 6 模組之 WHAT / WHY / HOW / SO WHAT 四層筆記）。
10. **書面報告與口頭簡報**：整合前述文件為期末報告，並製作簡報投影片，展示架構設計、移植流程、下板結果、ISA test 驗證。

5 Expected Deliverables

5.1 Hardware Demonstration

- 一張可程式化之 ZedBoard bitstream（`zedboard_top.bit`），上電即可執行 C 程式
- UART 串列輸出 `"Hello ZedBoard from tinyriscv!"` 字串
- LD2（`over`）與 LD3（`succ`）LED 點亮表示測試通過
- ISA compliance test 於下板環境之執行結果

5.2 Technical Documentation

- `README.md` — 專案總覽
- `ARCH_OVERVIEW.md` — tinyriscv 架構完整說明
- `PORTING_GUIDE.md` — Artix-7 → ZedBoard 移植 step-by-step 指南
- `BRINGUP_LOG.md` — 下板除錯過程與 gotcha 紀錄
- `RTL_WALKTHROUGH.md` — 核心模組深度筆記（WHAT / WHY / HOW / SO WHAT）
- 期末書面報告與口頭簡報投影片

5.3 RTL Source Code and Verification Artifacts

- 原樣保存之 upstream Verilog（`rtl_upstream/`）
- 自行撰寫之 wrapper、XDC、自訂 peripheral（`rtl_zedboard/`）
- 可重現之 Vivado project（`vivado_project/`）
- Icarus Verilog testbench 與模擬波形（`sim/`）

6 Development Environment and Tools

使用之工具與版本整理如 Table 6。

Table 6: Development Environment and Tool Versions.

Category	Tool	Version
FPGA Design	Xilinx Vivado WebPACK	2019.1
Target Hardware	Avnet ZedBoard (`xc7z020clg484-1`)	Rev C / D
Embedded SDK (Phase C)	Xilinx SDK	2019.1
RTL Simulation	Icarus Verilog + GTKWave	latest stable
RISC-V Toolchain	riscv32-unknown-elf-gcc	v9.2
Serial Terminal	MobaXterm / PuTTY	—
ASIC Synthesis (Phase C)	Synopsys Design Compiler + CBDK v2.1	系所伺服器
Version Control	Git + GitHub	—

References

[1] Blue Liang, "tinyriscv: A Beginner-Friendly Open-Source RISC-V Core," GitHub, https://github.com/liangkangnan/tinyriscv, 2020.

[2] Blue Liang, "從零開始寫 RISC-V 處理器," CSDN Blog, https://blog.csdn.net/weixin_42905573/article/details/107995202, 2020.

[3] RISC-V Foundation, *The RISC-V Instruction Set Manual, Vol. I: Unprivileged ISA*, Version 20191213, 2019.

[4] Avnet, *ZedBoard (Zynq-7000) Hardware User Guide*, Version 2.2.

[5] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2020.

[6] L. H. Crockett *et al.*, *The Zynq Book*, Strathclyde Academic Media, 2014.

[7] Xilinx, *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator (UG994)*, 2019.